

Kingdom of Saudi Arabia
Ministry of Education
Imam Abdulrahman bin Faisal University
College of Science and Humanities – Jubail

Growing Together

Community Afforestation & Tree Health System

CSC 236: Object Oriented Programming 2 – Final Project Report

Course	CSC 236: Object Oriented Programming 2
Instructor	Dr. Nora
Section	Section 1

Team Members

#	Student Name	Student ID
1	Fajr Hamad ALassaf	2250000194
2	Fatimah Altaher	2250002084
3	Layal Khalid Altamimi	2250006739
4	Razan Hafiz Sahli	2250005885
5	Shaden Daifallah algamdi	2250004960

Introduction

Growing Together is a Java-based desktop application developed to support community afforestation efforts and tree health monitoring. The system was built using Object-Oriented Programming (OOP) principles and connected to a MySQL database to store and manage all records efficiently.

The application enables users to manage afforestation campaigns, track planted trees, monitor tree health conditions, and coordinate volunteer activities through a graphical user interface (GUI). The system includes a login system for authentication, CRUD operations for all entities, and import/export functionality for tree and campaign data.

The project aims to address the lack of structured greenery management in urban environments by providing a centralized digital tool that synchronizes volunteer efforts with campaign goals.

Problem Statement

- **Environmental Challenges:** Rapid urban growth lacks structured greenery management and centralized coordination.
- **High Mortality Rates:** Planted trees often fail over time due to a complete lack of systematic health monitoring.
- **Inefficient Coordination:** Absence of a unified digital tool to sync community volunteer efforts with strategic campaign goals.

Objectives

- Build a community afforestation management application using Java.
- Apply Object-Oriented Programming concepts including inheritance and encapsulation.
- Connect the application with a MySQL database using JDBC.
- Create a user-friendly graphical interface using Java Swing.
- Allow users to add, display, update, and delete campaigns, trees, and locations.
- Implement a login and registration system with role-based access (Admin / Volunteer).
- Track tree health status (Healthy, Dead, Not Watered) over time.
- Export and import tree and campaign data using CSV files.

System Features

The Growing Together system provides the following core features:

Login and Registration

The system allows users to create new accounts and log in before accessing the dashboard. User authentication is implemented using the UserService class and database validation. The system supports two roles: Admin and Volunteer.

Campaign Management

- Add new afforestation campaigns with title, date, target tree count, and location.
- View all campaigns in a table with supervisor name and status.
- Update campaign status (Active / Completed / Cancelled).
- Delete campaigns from the system.
- Export campaign data to CSV.

Tree Management

- Add tree records with type, quantity, campaign, and location.
- View all planted trees with health status.
- Update tree health status (Healthy, Dead, Not Watered).
- Delete tree records.
- Export and import tree data using CSV files.

Location Management

- View all registered locations with name and city.
- Locations are linked to campaigns and trees.

Volunteer Management

Volunteers can register, log in, and participate in campaigns. The system tracks volunteer registrations per campaign and their planting activities.

System Design

GUI Design

The system was developed using Java Swing to create all graphical user interfaces. The application follows a consistent visual design with an olive-green color scheme throughout all screens.

Login Interface

The login screen allows users to enter their email and password. If the credentials are correct, the system redirects the user to the appropriate dashboard based on their role. A 'New User? Register' button is available for new users.

Register Interface

The registration screen allows new users to create an account by entering their first name, last name, email, password, phone number, and selecting a role (Admin or Volunteer).

Dashboard Interface

The dashboard is the main navigation screen. It displays the logged-in user's name and contains buttons for: View Campaigns, View Trees, View Locations, and Logout.

Campaign Interface

This interface displays all campaigns in a table showing ID, Title, Date, Target, Status, Supervisor Name, and Location ID. Available actions: Add Campaign, Update Status, Delete Campaign, View Trees, Export to CSV, and Import from CSV.

Tree Interface

This interface displays all trees in a table showing ID, Tree Type, Quantity, Planted By Name, Campaign ID, Location ID, and Health Status. Available actions: Add Tree, Update Health Status, Delete Tree, Export to CSV, and Import from CSV.

Location Interface

This interface displays all registered locations with their ID, Location Name, and City.

GUI Screens Summary

Screen	Description
Login	Email and password authentication
Register	New account creation with role selection

Dashboard	Main navigation menu with user name display
View Campaigns	Campaign table with CRUD operations and export
Add Campaign	Form to create a new campaign
Update Status	Dialog to change campaign status
View Trees	Tree table with health tracking and export/import
Add Tree	Form to add a new tree record
Update Health Status	Dialog to update tree health condition
View Locations	Read-only table of all locations

Class Hierarchy (Object-Oriented Programming)

The system applies Object-Oriented Programming principles including inheritance, encapsulation, and abstraction. The User class serves as the superclass for both Admin and Volunteer subclasses.

Superclass – User

The User class is the base class that stores common attributes shared by all system users.

Attribute	Type	Description
userID	int	Unique identifier for the user
firstName	String	User's first name
lastName	String	User's last name
email	String	Login email address
password	String	Account password
phone	String	Contact phone number
role	String	User role: Admin or Volunteer

Subclasses

- **Admin:** Extends User. Has full access to campaign and tree management, including add, update, delete, export, and import operations.
- **Volunteer:** Extends User. Can view campaigns and register for participation. Volunteers are linked to trees they have planted.

Other Classes

Class	Description
Tree	Stores tree type, quantity, campaign, location, and planting user
Campaign	Stores title, date, target, status, supervisor, and location

Location	Stores location name and city
TreeStatus	Logs health status reports for trees with date and reporter
VolunteerRegistration	Links volunteers to campaigns they registered for
UserProfile	Stores additional user profile data including bio and points
VolunteerSkill	Stores skills associated with a volunteer user

Note: The UML Class Diagram showing all class relationships, attributes, and methods is attached at the end of this report.

Database Design

The system is connected to a MySQL database named greenpath_db. The database stores all application data in relational tables connected through foreign keys.

Tables

Users Table

Column	Type	Constraint
userID	INT	PRIMARY KEY
firstName	VARCHAR(50)	NOT NULL
lastName	VARCHAR(50)	NOT NULL
email	VARCHAR(100)	UNIQUE, NOT NULL
password	VARCHAR(100)	NOT NULL
phone	VARCHAR(20)	NOT NULL
role	ENUM('Admin','Volunteer')	NOT NULL

Locations Table

Column	Type	Constraint
locationID	INT	PRIMARY KEY
name	VARCHAR(100)	NOT NULL
city	VARCHAR(100)	NOT NULL

Campaigns Table

Column	Type	Constraint
campaignID	INT	PRIMARY KEY

title	VARCHAR(200)	NOT NULL
date	DATE	NOT NULL
target	INT	NOT NULL
status	ENUM('Active','Completed','Cancelled')	DEFAULT 'Active'
supervisorID	INT	FOREIGN KEY → users(userID)
locationID	INT	FOREIGN KEY → locations(locationID)

Trees Table

Column	Type	Constraint
treeID	INT	PRIMARY KEY
treeType	VARCHAR(100)	NOT NULL
quantity	INT	NOT NULL
plantedBy	INT	FOREIGN KEY → users(userID)
campaignID	INT	FOREIGN KEY → campaigns(campaignID)
locationID	INT	FOREIGN KEY → locations(locationID)

Tree Status Table

Column	Type	Constraint
logID	INT	PRIMARY KEY
treeID	INT	FOREIGN KEY → trees(treeID)
reportedBy	INT	FOREIGN KEY → users(userID)
reportedDate	DATE	NOT NULL
healthStatus	VARCHAR(50)	NOT NULL

Volunteer Registrations Table

Column	Type	Constraint
registrationID	INT	PRIMARY KEY
campaignID	INT	FOREIGN KEY → campaigns(campaignID)
volunteerID	INT	FOREIGN KEY → users(userID)
registeredAt	DATE	NOT NULL

Database Relationships

- Each Campaign is supervised by one User (Admin) and located in one Location.
- Each Tree is planted by one User (Volunteer), belongs to one Campaign, and is in one Location.
- Each TreeStatus log is linked to one Tree and reported by one User.
- Each VolunteerRegistration links one Volunteer to one Campaign.

Note: The Entity-Relationship Diagram (ERD) is attached at the end of this report.

Data Access Tools and Techniques

The project uses JDBC (Java Database Connectivity) to connect the Java application with MySQL. The DatabaseConnection class manages the database connection using the Singleton pattern.

Database Connection

The DatabaseConnection class is responsible for establishing and maintaining the connection to the MySQL database using the URL, username, and password credentials. PreparedStatement is used to safely execute all SQL commands and prevent SQL injection.

CRUD Operations

Operation	Description	Tables Affected
Create (INSERT)	Add new records for campaigns, trees, locations, and users	users, campaigns, trees, locations
Read (SELECT)	Retrieve and display records in the GUI tables	All tables
Update (UPDATE)	Modify campaign status and tree health status	campaigns, tree_status
Delete (DELETE)	Remove campaign or tree records from the database	campaigns, trees

Login Facility

The system implements a multi-user login system. Users are stored in the database with their roles. Upon login, the system verifies credentials and redirects the user to the appropriate interface based on their role (Admin or Volunteer).

Service Classes

The application follows a service-layer architecture to separate database logic from the GUI. Service classes implemented include:

- UserService – handles login, registration, and user queries
- TreeService – manages tree CRUD operations
- CampaignService – manages campaign CRUD and status updates
- LocationService – retrieves location records

- TreeStatusService – handles health status reporting

Error Handling

The system handles user errors through GUI validation and Java exception handling mechanisms.

GUI Validation

- Empty fields are not accepted in any form.
- Quantity fields only accept valid integer values.
- Location is selected from a dropdown to prevent invalid entries.
- Campaign ID must exist in the database before performing delete or update operations.
- Confirmation dialogs are shown before destructive operations (e.g., logout, delete).

Exception Handling

The system uses try-catch blocks throughout the application to handle:

- Database connection errors (SQLException)
- Invalid input values (NumberFormatException)
- File I/O errors during CSV export/import (IOException)
- General unexpected errors (Exception)

All error messages are displayed to the user via JOptionPane dialogs with a clear description of the issue.

Data Export / Import

The system includes full export and import functionality for both tree and campaign records.

Export to CSV

Users can export tree or campaign data displayed in the table to a CSV file by clicking the 'Export to CSV' button. A file chooser dialog allows the user to select the save location and file name. The exported file contains all visible table columns.

Import from CSV

Users can import tree records from an existing CSV file by clicking the 'Import from CSV' button. A file chooser dialog allows the user to select the CSV file. The system reads each line, parses the fields, and inserts the records into the database.

Implementation

The application was implemented using the following technologies:

Technology	Purpose
------------	---------

Java	Core programming language
Java Swing	GUI framework for all application windows
MySQL	Relational database management system
JDBC	Java-to-database connectivity
NetBeans IDE	Development environment

Code Snippets

Database Connection

Establishes a single shared connection to the MySQL database using the Singleton pattern.

DatabaseConnection.java

```

        public class DatabaseConnection {
private static final String URL = "jdbc:mysql://localhost:3306/greenpath_db";
        private static final String USERNAME = "root";
        private static final String PASSWORD = "System";
        private static Connection connection = null;

        public static Connection getConnection() {
            if (connection == null) {
                try {
                    Class.forName("com.mysql.cj.jdbc.Driver");
                    connection = DriverManager.getConnection(URL, USERNAME, PASSWORD);
                } catch (ClassNotFoundException e) {
                    e.printStackTrace();
                } catch (SQLException e) {
                    e.printStackTrace();
                }
            }
            return connection;
        }
    }

```

Login

Authenticates the user and redirects to the appropriate dashboard based on their role.

LoginForm.java

```

        String email    = emailField.getText().trim();
        String password = new String(passwordField.getPassword()).trim();
        User user = UserService.login(email, password);

        if (user != null) {
            if (user instanceof Admin) {
                openAdminDashboard((Admin) user);
            } else {
                openVolunteerDashboard((Volunteer) user);
            }
        } else {
            JOptionPane.showMessageDialog(this, "Invalid email or password!");
        }
    }

```

Inheritance – Superclass & Subclasses

Demonstrates the class hierarchy where Admin and Volunteer both extend the User superclass.

User.java / Admin.java / Volunteer.java

```

        public class User {
            private int    userID;
            private String firstName;
            private String lastName;
        }

        public class Admin extends User {
            public Admin(int userID, String firstName, String lastName,
                String email, String password, String phone) {
                super(userID, firstName, lastName, email, password, phone, "Admin");
            }
        }

        public class Volunteer extends User {
            public Volunteer(int userID, String firstName, String lastName,
                String email, String password, String phone) {
                super(userID, firstName, lastName, email, password, phone, "Volunteer");
            }
        }

```

CRUD Operations

Covers the four core database operations: create, read, update, and delete.

Create – TreeService.plantTree()

```

        boolean success = TreeService.plantTree(
            treeID, treeType, quantity,
            admin.getUserID(), campaignID, locationID
        );

```

Read – TreeService.getAllTrees()

```

        List<Tree> trees = TreeService.getAllTrees();
        for (Tree t : trees) {
            model.addRow(new Object[]{
                t.getTreeID(), t.getTreeType(), t.getQuantity()
            });
        }

```

Update – TreeStatusService.reportTreeStatus()

```

        boolean success = TreeStatusService.reportTreeStatus(
            treeID, admin.getUserID(), newStatus
        );

```

Delete – TreeService.deleteTree()

```

        boolean success = TreeService.deleteTree(treeID);

```

Error Handling

Handles invalid user input using try-catch blocks to prevent application crashes.

Input Validation

```

        try {
            int quantity = Integer.parseInt(quantityField.getText().trim());
        } catch (NumberFormatException ex) {
            JOptionPane.showMessageDialog(this, "Please enter valid numbers!");
        } catch (Exception ex) {
            JOptionPane.showMessageDialog(this, "Error: " + ex.getMessage());
        }

```

Export to CSV

Exports all tree records to a CSV file selected by the user.

exportToCSV()

```
private void exportToCSV() {
    try (PrintWriter writer = new PrintWriter(new File("trees_export.csv"))) {
        for (int row = 0; row < treestable.getRowCount(); row++) {
            for (int col = 0; col < treestable.getColumnCount(); col++) {
                writer.print(treestable.getValueAt(row, col));
            }
            if (col < treestable.getColumnCount() - 1) writer.print(",");
            writer.println();
        }
    } catch (IOException ex) {
        JOptionPane.showMessageDialog(this, "Export failed!");
    }
}
```

Import from CSV

Reads tree records from a CSV file and inserts them into the database.

importFromCSV()

```
private void importFromCSV() {
    try (BufferedReader reader =
        new BufferedReader(new FileReader("trees_export.csv"))) {
        String line;
        while ((line = reader.readLine()) != null) {
            String[] data = line.split(",");
            int treeID = Integer.parseInt(data[0]);
            String treeType = data[1];
            TreeService.plantTree(treeID, treeType,
                Integer.parseInt(data[2]), Integer.parseInt(data[3]),
                Integer.parseInt(data[4]), Integer.parseInt(data[5]));
        }
    } catch (Exception ex) {
        JOptionPane.showMessageDialog(this, "Import failed!");
    }
}
```

Testing

The system was tested to ensure all functionalities work correctly. The following test cases were executed:

Test Case	Input	Expected Result	Actual Result
TC1: User Registration	Valid name, email, password, phone, role	Account created successfully	Passed
TC2: Login – Valid	Correct email and password	Dashboard opens with user name	Passed
TC3: Login – Invalid	Wrong email or password	Error message displayed	Passed
TC4: Add Campaign	Valid campaign data with location	Campaign saved in database	Passed
TC5: Update Campaign Status	Existing campaign ID, new status	Status updated successfully	Passed

TC6: Delete Campaign	Valid campaign ID	Campaign deleted from database	Passed
TC7: Add Tree	Valid tree type, quantity, campaign ID	Tree saved in database	Passed
TC8: Update Tree Health	Select tree, choose new status	Health status updated	Passed
TC9: Delete Tree	Valid campaign ID and tree ID	Tree deleted from database	Passed
TC10: Export to CSV	Click Export, choose file path	CSV file created with all records	Passed
TC11: Import from CSV	Click Import, select CSV file	Records loaded into database	Passed
TC12: Invalid Input	Non-numeric quantity value	Error dialog shown, no crash	Passed
TC13: Delete Non-existent ID	Invalid campaign ID (e.g. 3011)	'Not Found' error message shown	Passed
TC14: Logout	Click Logout, confirm dialog	User redirected to login screen	Passed

Challenges Faced

- **Logic Implementation:** Building a robust hierarchical OOP structure to connect complex entities like Trees, Campaigns, Locations, and Volunteers.
- **GUI Navigation:** Designing fluid, thread-safe transitions between supervisor configuration dashboards and record tracking interfaces.
- **Data Lifecycle:** Handling data persistence across multiple tracking sessions without losing health status updates.
- **Database Connectivity:** Establishing and maintaining a stable JDBC connection to MySQL across all service classes.
- **CSV Import Validation:** Ensuring imported CSV records match the expected format and handling malformed data gracefully.

Resolutions

- **Encapsulation & OOP:** Successfully separated backend business logic from frontend GUI components to achieve modular and maintainable code.
- **Java Collections API:** Utilized ArrayLists and Maps to handle real-time volunteer, tree, and campaign queries efficiently.
- **Singleton Pattern:** Used a single shared DatabaseConnection instance to avoid multiple redundant connections.
- **Service Layer:** Implemented dedicated service classes for each entity to centralize all database operations.

Conclusion

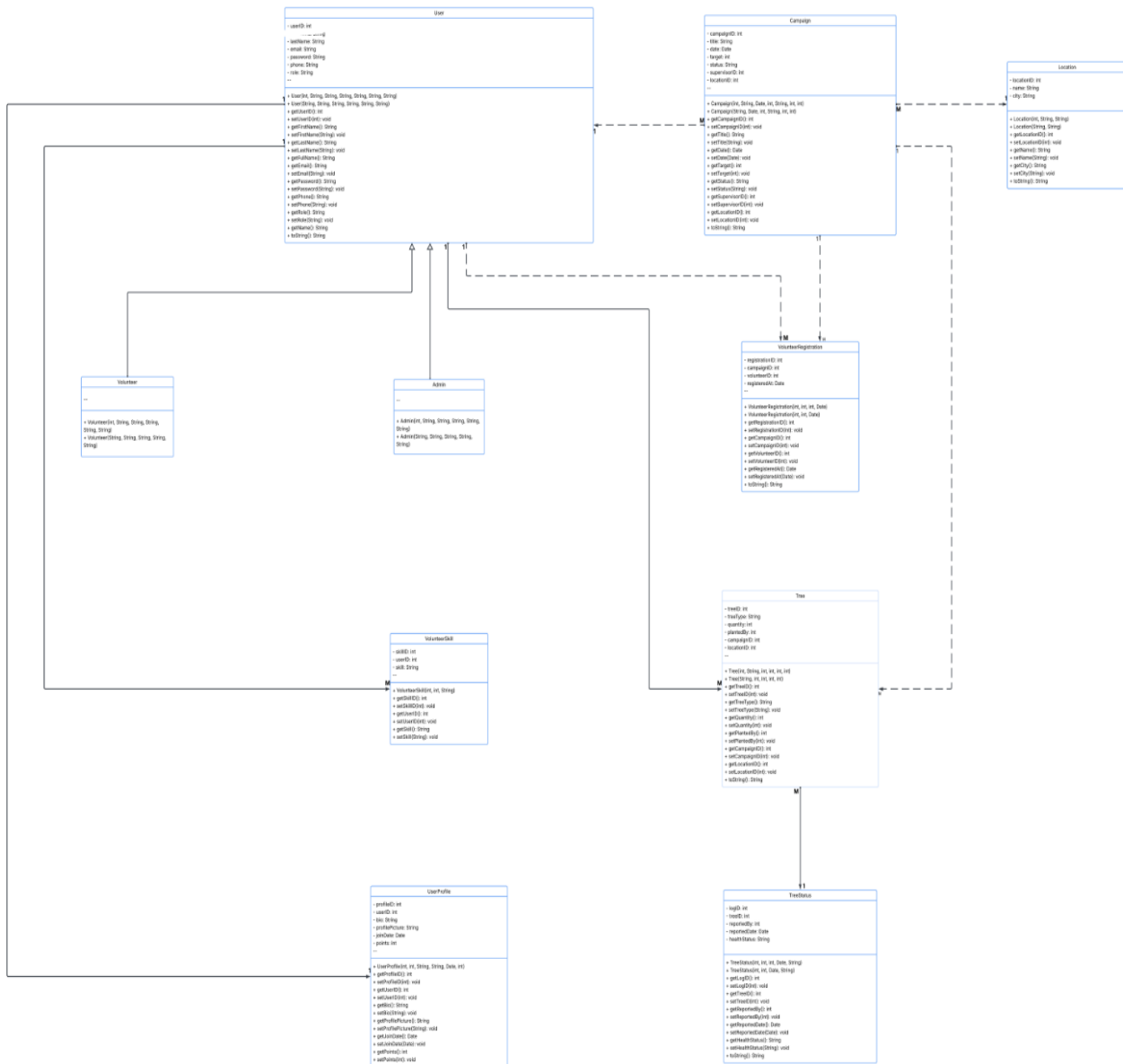
The Growing Together system successfully achieved its objectives by providing a complete desktop application for managing community afforestation campaigns and tree health monitoring. The system applies Object-Oriented Programming concepts including inheritance, encapsulation, and service-layer architecture, and uses JDBC to connect to a MySQL database.

The application enables users to manage campaigns, trees, and locations through easy-to-use graphical interfaces. It includes login authentication with role-based access, full CRUD operations, input validation, exception handling, and CSV export/import functionality.

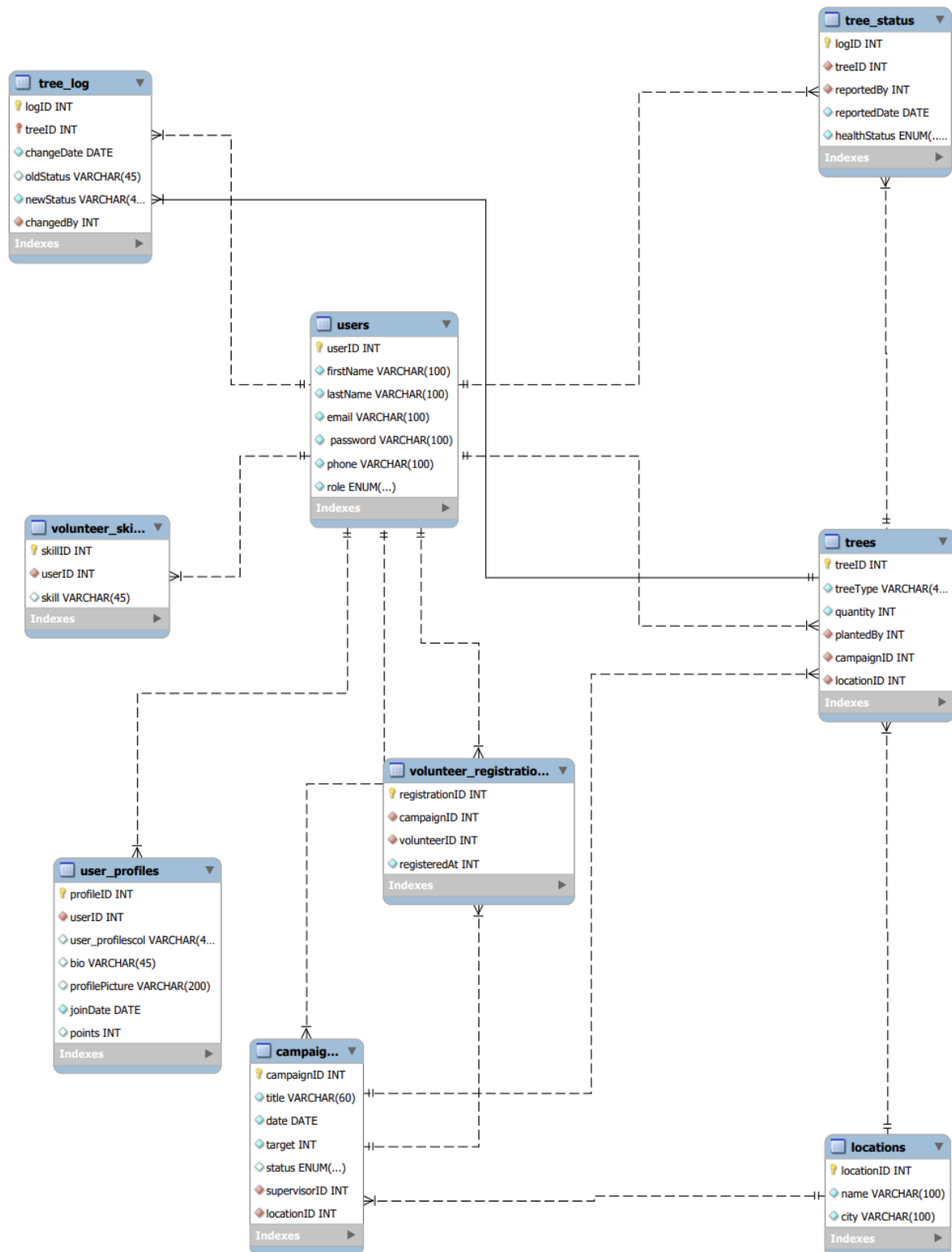
Future Improvements

- Adding a search and filter feature for campaigns and trees.
- Implementing a reporting dashboard with statistics and charts.
- Enhancing database security with password hashing.
- Adding email notification support for campaign updates.
- Extending the volunteer profile system with skill tracking and point rewards.
- Developing a mobile companion application.

UML



ERD



GUI

Tree Campaign - Register

Create New Account

First Name:

Last Name:

Email:

Password:

Phone:

Role: Volunteer

Green Community Tree - Login

Green Tree Community

Login to your account

Email:

Password:

View Trees

ID	Tree Type	Quantity	Planted By Name	Campaign ID	Location ID	Health Status
4001	Sidr Tree					Healthy
4002	Olive Tree					Healthy
4003	Palm Tree					Not Watered
4004	Acacia Tree					Dead
4005	Ghaf Tree					Not Watered
4006	Jacaranda					Healthy
4007	Bougainvillea					Dead
4008	Frangipani					Healthy
4009	Moringa Tree					Healthy

Tree Campaign System

Green Tree Community

Admin: Fajr Al-Assaf

View Campaigns

ID	Title	Supervisor Name	Location ID
3000	Riyadh Greening Initia...	Fajr Al-Assaf	2000
3001	Jeddah Coastal Cleanup	Layal Al-tamimi	2001
3002	Dammam Forest Project	Shaden Al-ghamdi	2003
3003	Abha Mountain Planting	Fajr Al-Assaf	2004
3004	Taif Rose Revival	Layal Al-tamimi	2005
3005	Yanbu Waterfront Trees	Shaden Al-ghamdi	2006
3006	Al-Ula Heritage Forest	Fajr Al-Assaf	2007
3007	Khobar Palm Project	Layal Al-tamimi	2008
3008	Hail Desert Bloom	Shaden Al-ghamdi	2009
3009	Year End Mega Campaign	Fajr Al-Assaf	2000
3010	Eastern Province Gree...	Fajr Al-Assaf	2008

Add New Campaign

Create New Campaign

Title:

Date (YYYY-MM-DD):

Target (trees):

Location: 2000 - King Abdullah Park ...

Campaign ID:

View Campaigns

Back to Main Menu

ID	Title	Date	Target	Status	Supervisor Name	Location ID
3000	Riyadh Greening Initia...	2025-01-15	500	Completed	Fajr Al-Assaf	2000
3001	Jeddah Coastal Cleanup	2025-02-20	1000	Completed	Layal Al-tamimi	2001
3002	Dammam Forest Project	2025-03-10	750	Completed	Shaden Al-ghamdi	2003
3003	Abha Mountain Planting	2025-04-05	800	Active	Fajr Al-Assaf	2004
3004	Taif Rose Revival	2025-05-12	600	Active	Layal Al-tamimi	2005
3005	Yanbu Waterfront Trees	2025-06-18	900	Active	Shaden Al-ghamdi	2006
3006	Al-Ula Heritage Forest	2025-07-22	400	Active	Fajr Al-Assaf	2007
3007	Khobar Palm Project	2025-08-30	700	Active	Layal Al-tamimi	2008
3008	Hail Desert Bloom	2025-09-14	300	Active	Shaden Al-ghamdi	2009
3009	Year End Mega Campaign	2025-12-01	2000	Completed	Fajr Al-Assaf	2000
3010	Eastern Province Gree...	2025-10-15	1500	Active	Fajr Al-Assaf	2008

Add Campaign Update Status Delete Campaign View Trees Export to CSV Import from CSV

View Campaigns

Back to Main Menu

ID	Title	Date	Target	Status	Supervisor Name	Location ID
3000	Riyadh Greening Initia...	2025-01-15	500	Completed	Fajr Al-Assaf	2000
3001	Jeddah Coastal Cleanup	2025-02-20	1000	Completed	Layal Al-tamimi	2001
3002	Dammam Forest Project	2025-03-10	750	Completed	Shaden Al-ghamdi	2003
3003	Abha Mountain Planting	2025-04-05			Fajr Al-Assaf	2004
3004	Taif Rose Revival	2025-05-12			Layal Al-tamimi	2005
3005	Yanbu Waterfront Trees	2025-06-18			Shaden Al-ghamdi	2006
3006	Al-Ula Heritage Forest	2025-07-22			Fajr Al-Assaf	2007
3007	Khobar Palm Project	2025-08-30	700	Active	Layal Al-tamimi	2008
3008	Hail Desert Bloom	2025-09-14	300	Active	Shaden Al-ghamdi	2009
3009	Year End Mega Campaign	2025-12-01	2000	Completed	Fajr Al-Assaf	2000
3010	Eastern Province Gree...	2025-10-15	1500	Active	Fajr Al-Assaf	2008

Not Found
Campaign ID 3011 not found!

Add Campaign Update Status Delete Campaign View Trees Export to CSV

View Locations

Back to Main Menu

ID	Location Name	City
2000	King Abdullah Park	Riyadh
2001	Al-Mamzar Beach Park	Jeddah
2002	Al-Shallal Theme Park	Jeddah
2003	King Fahd Park	Dammam
2004	Al-Salam Park	Abha
2005	Al-Faisaliah Garden	Taif
2006	Corniche Park	Yanbu
2007	Al-Ula Heritage Site	Al-Ula
2008	Palm Beach Resort	Al-Khobar
2009	Desert Rose Garden	Hail

View Campaigns

Back to Main Menu

ID	Title	Date	Target	Status	Supervisor Name	Location ID
3000	Riyadh Greening Initia...	2025-01-15	500	Completed	Fajr Al-Assaf	2000
3001	Jeddah Coastal Cleanup	2025-02-20	1000	Completed	Layal Al-tamimi	2001
3002	Dammam Forest Project	2025-03-10	750	Completed	Shaden Al-ghamdi	2003
3003	Abha Mountain Planting	2025-04-05	800	Active	Fajr Al-Assaf	2004
3004	Taif Rose Revival	2025-			Layal Al-tamimi	2005
3005	Yanbu Waterfront Trees	2025-			Shaden Al-ghamdi	2006
3006	Al-Ula Heritage Forest	2025-			Fajr Al-Assaf	2007
3007	Khobar Palm Project	2025-			Layal Al-tamimi	2008
3008	Hail Desert Bloom	2025-09-14	300	Active	Shaden Al-ghamdi	2009
3009	Yamalo Madaen Campaign	2025-10-20	1200	Completed	Fajr Al-Assaf	2000
3010	Eastern Province Gree...	2025-10-15	1500	Active	Fajr Al-Assaf	2008

Add Campaign

Update Status

Delete Campaign

View Trees

Export to CSV

Update Status

Select new status for: Year End Mega Campaign

Active

OK

Cancel

View Trees

Back to Main Menu

ID	Tree Type	Quantity	Planted By Name	Campaign ID	Location ID	Health Status
4001	Sidr Tree	85	Fatima Al-Taher	3000	2001	Healthy
4002	Olive Tree	200	Nora Al-Dosari	3001	2002	Healthy
4003	Palm Tree	150	Khalid Al-Harbi	3001	2003	Not Watered
4004	Acacia Tree	95	Fajr Al-Assaf	3002	2004	Dead
4005	Ghaf Tree	110			2005	Not Watered
4006	Jacaranda	65			2006	Healthy
4007	Bougainvillea	180			2007	Dead
4008	Frangipani	75			2008	Healthy
4009	Moringa Tree	130			2009	Healthy

Add Tree

Update Health Sta...

Delete Tree

Export to CSV

Enter Tree Details to Delete

Campaign ID: 3000

Tree ID: 4001

OK

Cancel